

Avance 2: Modelado de un sistema RF con corrección y detección de errores para una aplicación médica mediante un SoC nRF52832 Nordic Semiconductor

Alexandra Alfaro Elizondo*, Francela Arias Vargas[†] y Kendall Madrigal Campos[‡]

*Escuela de Ingeniería Electrónica, Instituto Tecnológico de Costa Rica (ITCR), 30101 Cartago, Costa Rica, {alexvaneska, fcelavar0802, kendallmacampos2941}@estudiantec.cr

Abstract—Este trabajo presenta el modelado de un sistema de comunicación RF con detección y corrección de errores orientado a aplicaciones médicas, basado en el SoC nRF52832. El sistema integra los módulos transmisor, canal y receptor. En el transmisor se implementaron los códigos Hamming en Python y la modulación GFSK conforme al estándar Bluetooth Low Energy (BLE). El canal se modeló mediante ruido AWGN y el receptor incorporó procesos de demodulación y decodificación. Los resultados confirman la correcta integración de los bloques, evidenciando una transmisión robusta, eficiente y adecuada para operación en tiempo real.

Keywords—Bluetooth Low Energy, codificación Hamming, corrección de errores, GFSK, SoC nRF52832

I. INTRODUCCIÓN

El desarrollo de sistemas de comunicación inalámbricos confiables es fundamental en aplicaciones médicas donde la integridad y disponibilidad de los datos resultan críticas para la seguridad del paciente. En este contexto, el presente trabajo aborda el modelado de un sistema de comunicación RF con detección y corrección de errores basado en el SoC nRF52832 de Nordic Semiconductor, orientado a la transmisión de información fisiológica obtenida durante una prueba de esfuerzo cardíaco (*Ecostress*). El proyecto integra conocimientos de teoría de la información, modulación digital y procesamiento de señales para construir un sistema robusto y eficiente que opere en tiempo real.

El sistema propuesto se divide en tres módulos principales: transmisor (TX), canal y receptor (RX). En el módulo transmisor se implementaron las etapas de codificación y modulación. La codificación de canal se realizó mediante los esquemas Hamming (7,4) y (15,11), que permiten detectar y corregir errores de un bit con baja complejidad y mínima redundancia. La modulación empleada corresponde a la técnica GFSK (*Gaussian Frequency Shift Keying*), utilizada en el estándar Bluetooth Low Energy (BLE), seleccionada por su continuidad de fase, bajo consumo energético y alta eficiencia espectral.

El canal se modeló mediante una fuente de ruido aditivo blanco gaussiano (AWGN), mientras que el receptor incluye los procesos de demodulación y decodificación, además de

una etapa de visualización que permite comparar los datos transmitidos con los originales. Los resultados experimentales validan la correcta integración entre los bloques y evidencian el cumplimiento de los requerimientos de confiabilidad, eficiencia y operación en tiempo real, consolidando una base sólida para futuras implementaciones en hardware médico embebido.

II. METODOLOGÍA DE DESARROLLO

A. Módulo transmisor (Tx)

1) Bloque Latidos por minuto/Prueba Ecostress

El bloque de Ecostress tuvo como propósito caracterizar la señal de latidos por minuto (BPM) proveniente del monitor cardíaco durante la prueba de esfuerzo y verificar la naturaleza estadística de los datos antes de aplicar procesos de filtrado. Se trabajó con el archivo *Dataset.xlsx*, conteniendo un total de 127 muestras tomadas cada 10 segundos. La información fue cargada en Python mediante las librerías *pandas* y *numpy*, garantizando un tratamiento uniforme de los valores numéricos y su posterior análisis.

Seguidamente se calcularon métricas descriptivas tales como la media, desviación estándar, valores extremos y cuantiles, con el fin de evaluar la tendencia central y la dispersión de la variable BPM. Estos parámetros sirvieron como base para comparar el comportamiento del conjunto antes y después del preprocesamiento.

Para evaluar la forma de la distribución se aplicó la prueba de normalidad de Anderson-Darling utilizando la función *anderson* del paquete *scipy.stats*. La hipótesis nula (H_0) estableció que los datos provienen de una distribución normal, mientras que la hipótesis alternativa (H_1) asumía lo contrario. El contraste se realizó con un nivel de significancia de $\alpha = 0.05$ (95 % de confianza), comparando el estadístico A^2 contra los valores críticos correspondientes.

Como apoyo visual, se generaron un histograma de densidad y un diagrama de cajas (*boxplot*) para identificar posibles sesgos, dispersión y valores atípicos. La detección de valores atípicos se complementó mediante la implementación del test

de Grubbs bilateral, aplicado de forma iterativa hasta no encontrar elementos que superaran el umbral crítico G_{crit} , calculado a partir de la distribución t de Student.

2) Bloque Preprocesamiento

El bloque de preprocesamiento tuvo como objetivo principal reducir el ruido presente en la señal de latidos por minuto proveniente del bloque Ecostress, con el fin de preservar las características fisiológicas relevantes antes del proceso de codificación. Todas las pruebas se desarrollaron en Python dentro del entorno *Jupyter Notebook*, empleando las librerías *NumPy*, *Pandas*, *SciPy* y *Matplotlib*.

En primer lugar, se cargó el conjunto de datos original desde el archivo *Dataset.xlsx*, el cual contiene 127 mediciones tomadas cada 10 s. Los datos se graficaron para observar la tendencia global y el nivel de ruido inherente. La Figura 4 muestra la señal sin filtrar, donde se aprecia un crecimiento progresivo de la frecuencia cardíaca hasta el punto máximo de esfuerzo y una disminución posterior durante la fase de recuperación.

Posteriormente, se investigaron diferentes técnicas de filtrado digital orientadas al suavizado de señales unidimensionales con ruido aleatorio. Entre los métodos considerados se incluyeron los filtros de Media Móvil, Mediana, Savitzky-Golay y Kalman. A partir de la revisión bibliográfica y de las pruebas exploratorias, se construyó una matriz comparativa tipo *trade-off* que permitió evaluar los métodos en función de los siguientes criterios técnicos: eficacia de filtrado, preservación de información, tiempo de cómputo y complejidad de implementación.

Con base en dicha matriz, se seleccionaron los filtros de Mediana y Savitzky-Golay para una comparación más detallada, ya que ambos ofrecían una alta capacidad de atenuación del ruido y una implementación práctica en el entorno del sistema ECOSTRESS. Para el filtro de Mediana se utilizó una ventana de cinco muestras (*kernel_size=5*), mientras que para el filtro de Savitzky-Golay se estableció una ventana de cinco muestras y un polinomio de orden dos (*window_length=5*, *polyorder=2*).

El tiempo de ejecución de cada método se midió mediante el módulo `time.perf_counter()` de Python, ejecutando 200 iteraciones consecutivas para cada algoritmo a fin de calcular un promedio representativo. Dichas mediciones permitieron verificar el cumplimiento del *deadline* de 10 s definido en el proyecto para el procesamiento en tiempo real.

Finalmente, se almacenaron las señales filtradas y los resultados estadísticos en archivos de salida dentro de la carpeta *Documentacion/Avance 1/*, junto con las gráficas y los datos intermedios generados.

3) Bloque de Codificación

La codificación de canal se implementó en Python mediante la librería *NumPy*, con el objetivo de generar y analizar el desempeño de los códigos Hamming (7,4) y (15,11). Estos códigos permiten la detección y corrección de un error de un solo bit mediante la incorporación de bits de paridad

calculados a partir de una matriz generadora G y una matriz de verificación de paridad H .

Para el caso del código (7,4), se utilizó la matriz generadora:

$$G_{(7,4)} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

mientras que el código (15,11) se construyó de forma análoga con una matriz $G_{(15,11)}$ de dimensiones 11×15 .

El flujo de ejecución se resume en los siguientes pasos:

- 1) Se generaron secuencias aleatorias de bits de información (`numpy.random.randint`).
- 2) Se codificaron utilizando las operaciones modulares con las matrices G y H .
- 3) Se simulaban errores aleatorios en uno o más bits para verificar la capacidad de corrección.
- 4) Los resultados se exportaron en archivos `.txt` para su uso posterior en MATLAB.

El diagrama general del proceso se ilustra en la Figura 1.

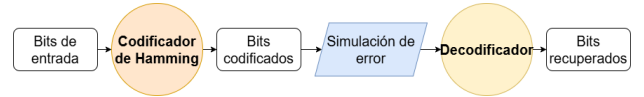


Fig. 1: Flujo de codificación Hamming en Python.

El código fue diseñado para evaluar el tiempo de ejecución, el número de bits generados, la tasa de redundancia y la precisión de corrección ante errores simulados.

4) Bloque de Modulación

La modulación se desarrolló en MATLAB/Octave en el script `modulacion.m`, que toma los bits codificados desde Python y los convierte en una señal modulada GFSK, siguiendo el estándar *Bluetooth Low Energy* (BLE).

Los principales parámetros de simulación se resumen en la Tabla I.

TABLE I: Parámetros utilizados en la simulación de modulación.

Parámetro	Símbolo	Valor
Tasa de bits	R_b	1 kbps
Muestras por bit	N_s	100
Frecuencia portadora	f_c	2 kHz
Desviación de frecuencia	Δf	1 kHz
Producto BT del filtro	BT	0.5
Índice de modulación	$h = 2\Delta f/R_b$	2

El flujo binario se normalizó al formato NRZ (± 1) y se aplicó un sobremuestreo de 100 muestras por bit. La modulación FSK básica se generó a partir de la frecuencia instantánea:

$$f_i(t) = f_c + \Delta f \cdot m(t)$$

donde $m(t)$ es la señal NRZ.

Para la modulación GFSK, se empleó un filtro gaussiano $g(t)$ con desviación estándar σ_s para suavizar las transiciones,

reduciendo la interferencia intersímbolo. La fase acumulada se obtuvo mediante:

$$\phi_{\text{GFSK}}(t) = \pi h \int g(t) * m(t) dt$$

y la señal transmitida:

$$s_{\text{GFSK}}(t) = \cos(2\pi f_c t + \phi_{\text{GFSK}}(t))$$

Las señales NRZ, FSK y GFSK se visualizaron sobre los primeros 10 bits transmitidos.

5) Antena

Durante la etapa de integración en el entorno *Autodesk Eagle* se realizó la incorporación de la antena SWRA092B proveniente de la librería *texas*. La antena fue ubicada en el borde superior del PCB y orientada con un ángulo de 90° hacia el exterior de la tarjeta, siguiendo las recomendaciones del fabricante para maximizar la eficiencia de radiación. El pad 1 de la antena se conectó directamente a la salida RF (OUT) del conector MM8130-2600, mientras que el pad 2 se unió al plano de tierra (GND) mediante una vía pasante que asegura una baja impedancia de retorno. El bloque antena tiene como función transmitir la señal de radiofrecuencia generada por el SoC nRF52832 hacia el medio, completando el enlace de comunicación *Bluetooth Low Energy* (BLE) del sistema, y el diseño se basa en la topología indicada en el PCB de referencia del curso, la cual incorpora el conector RF MM8130-2600 y la antena chip SWRA092B. Finalmente, se verificó la continuidad eléctrica de las señales OUT y GND mediante las herramientas *Show* y *Ratsnest*, confirmando que no existían conexiones pendientes (mensaje “*Nothing to do*”), lo que garantizó la correcta interconexión de la antena dentro del diseño.

B. Módulo canal

En este bloque se modeló el canal de comunicación del sistema RF mediante la adición de ruido gaussiano blanco aditivo (AWGN) generado a partir de la transformación de Box-Muller, la cual permite convertir números aleatorios uniformes en una distribución normal estándar $N(0,1)$. Este enfoque proporciona una representación estadística del ruido térmico presente en un canal real. Para garantizar la aleatoriedad en cada ejecución se alternó la semilla del generador de números aleatorios, obteniendo diferentes realizaciones del ruido.

C. Módulo Receptor

Se recomienda iniciar el diseño conceptual de la etapa de demodulación, modelando el esquema de modulación *Gaussian Frequency Shift Keying* (GFSK) utilizado por la tecnología *Bluetooth Low Energy* (BLE), conforme al estándar IEEE 802.15.1 [1].

El desarrollo puede realizarse en *Octave* o *Matlab* mediante la implementación de un demodulador no coherente que permita analizar el desempeño del sistema ante diferentes niveles de ruido aditivo blanco gaussiano (AWGN). Este modelo permitirá obtener la tasa de error de bit (BER) y verificar la robustez de la transmisión antes de integrar la etapa de decodificación.

De forma preliminar, se recomienda considerar un esquema de decodificación mediante códigos lineales de tipo Hamming, dado su bajo costo computacional y facilidad de implementación en sistemas embebidos, asegurando compatibilidad con el SoC nRF52832.

1) Bloque Demodulador

2) Bloque Decodificador

Para el bloque de decodificación se utilizó un enfoque basado en el código Hamming (15,11) implementado en Python. Se trabajó con dos archivos de entrada generados en el flujo del sistema: los bits codificados transmitidos (`bits_codificados_h1511.txt`) y los bits recibidos a la salida del demodulador (`gfsk_bits_demod.txt`). Debido a que el código opera con palabras de longitud $n = 15$, ambos vectores se alinearon recortándolos a una longitud común múltiplo de 15, evitando procesar palabras incompletas. Como referencia de desempeño previo a la corrección, se calculó la BER bruta comparando bit a bit las secuencias codificadas transmitidas y recibidas.

La decodificación se realizó mediante el cálculo del síndrome usando la matriz de comprobación de paridad H de dimensión 4×15 , definida de forma consistente con el codificador empleado. Para cada palabra recibida $\mathbf{r}$ se obtuvo el síndrome $\mathbf{s} = H \cdot \mathbf{r}^T \bmod 2$. Un síndrome nulo indicó ausencia de error, mientras que un síndrome no nulo asociado a una de las columnas de H permitió localizar y corregir un error de un solo bit invirtiendo la posición correspondiente. Posteriormente se extrajeron los 11 bits de información de cada palabra corregida para formar la secuencia decodificada de salida.

Para cuantificar la mejora introducida por el decodificador, los bits de información originales se reconstruyeron a partir de los bits codificados transmitidos, utilizando las mismas posiciones sistemáticas de información empleadas en el receptor. Con ello se calculó la BER a nivel de información comparando la secuencia original con la secuencia decodificada. Finalmente, los bits de información corregidos se guardaron en `bits_decodificados_h1511.txt` como salida del bloque, y de forma complementaria se registraron parámetros y métricas de desempeño (E_b/N_0 o SNR, errores y BER antes y después de la decodificación) en un archivo de resultados para el análisis del sistema.

3) Bloque Visualización

El bloque de visualización se implementó en *Python* sobre un cuaderno Jupyter, utilizando las bibliotecas NumPy, Pandas, Matplotlib y SciPy. Su objetivo es consolidar la información generada a lo largo de la cadena de transmisión (monitor-TX-canal-RX) y presentarla en forma de métricas y gráficas que permitan evaluar el desempeño del sistema de telemetría de frecuencia cardíaca.

Los datos de entrada provienen de dos archivos en formato `.xlsx`: `Dataset_Crudo.xlsx`, que contiene la señal de frecuencia cardíaca de referencia, y `Dataset.xlsx`, que contiene la señal reconstruida tras el paso por el enlace de

comunicaciones. En ambos casos se trabaja con dos columnas, el tiempo en segundos t y los latidos por minuto BPM. Adicionalmente, el bloque está preparado para leer resultados de tasa de error de bit (BER) desde archivos `.csv` generados por el bloque de canal/decodificador.

Clasificación en zonas de entrenamiento A partir de la edad del usuario (32 años) se calcula la frecuencia cardiaca máxima teórica como

$$FC_{\max} = 220 - \text{edad} = 188 \text{ bpm.}$$

Con esta referencia se definen las zonas de entrenamiento según el porcentaje de esfuerzo:

- Calentamiento: 50%–60% de $FC_{\max}$,
- Quema grasa: 60%–70%,
- Aeróbica: 70%–80%,
- Anaeróbica: 80%–90%,
- Extrema: 90%–100%.

Para cada muestra de la serie reconstruida $BPM_{rx}(t_i)$ se calcula la razón $r_i = BPM_{rx}(t_i)/FC_{\max}$ y se asigna una etiqueta discreta de zona mediante una función de clasificación. Posteriormente se agrupan las muestras contiguas que pertenecen a la misma zona, generando una tabla de segmentos con el tiempo de inicio, tiempo de fin, duración, y los valores mínimo y máximo de BPM en cada tramo. Esta tabla se exporta a un archivo CSV y se utiliza para generar el gráfico de zonas de entrenamiento coloreando cada muestra según su categoría.

Cálculo y representación de curvas BER El bloque de visualización contempla la lectura de resultados de BER para diferentes valores de relación señal a ruido (SNR) desde archivos `.csv` asociados a los esquemas de codificación Hamming (7, 4) y Hamming (15, 11). Cada archivo contiene, como mínimo, las columnas `snr_db` y `ber`, que se empaquetan en un `DataFrame` de `Pandas`.

Cuando no se dispone de datos experimentales, se genera una curva de referencia teórica para modulación BPSK en canal AWGN, modelada como

$$P_b(\text{SNR}_{\text{lin}}) = \frac{1}{2} \text{erfc}(\sqrt{\text{SNR}_{\text{lin}}}),$$

donde $\text{SNR}_{\text{lin}} = 10^{\text{SNR}_{\text{dB}}/10}$. A partir de estas muestras se construye la gráfica BER vs. SNR en escala semilogarítmica, agrupando por esquema de codificación para facilitar comparaciones futuras.

Comparación de BPM de referencia y reconstruidos. Para evaluar la calidad de la reconstrucción de la señal de frecuencia cardiaca, se comparan las series de referencia $BPM_{\text{ref}}(t)$ (archivo `Dataset_Crudo.xlsx`) y reconstruida $BPM_{rx}(t)$ (archivo `Dataset.xlsx`). Ambas señales se alinean temporalmente recortándolas a la longitud mínima común y se calculan dos métricas estándar:

- El error cuadrático medio (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2},$$

donde $y_i = BPM_{\text{ref}}(t_i)$ y $\hat{y}_i = BPM_{rx}(t_i)$.

- El sesgo porcentual (Pbias):

$$\text{Pbias} = 100 \frac{\sum_{i=1}^N (\hat{y}_i - y_i)}{\sum_{i=1}^N y_i} [\%].$$

Las dos curvas se representan de forma conjunta en el dominio del tiempo, incorporando los valores de RMSE y Pbias en el título de la figura.

Medición del tiempo de ejecución El coste computacional del bloque de visualización se estima mediante la función `time.perf_counter()`, que mide el tiempo transcurrido entre el inicio y el fin de una ejecución básica del bloque (clasificación de zonas y cálculo de métricas). Esta medición se repite $N = 200$ veces y se calcula el tiempo promedio y la desviación estándar:

$$t_{\text{prom}} = \frac{1}{N} \sum_{k=1}^N t_k, \quad \sigma_t = \sqrt{\frac{1}{N-1} \sum_{k=1}^N (t_k - t_{\text{prom}})^2}.$$

Con ello se cuantifica el impacto del bloque de visualización respecto al resto del sistema.

III. ANÁLISIS DE RESULTADOS

A. Módulo Transmisor(Tx)

1) Bloque Latidos por minuto/Prueba Ecotress

El conjunto de datos analizado corresponde a 127 mediciones de latidos por minuto (BPM) obtenidas durante una prueba de Ecotress. En la Tabla II se resumen las principales métricas descriptivas calculadas. La media de 129.16 BPM y una desviación estándar de 19.73 BPM reflejan una dispersión moderada de los valores, asociada a las variaciones fisiológicas esperadas durante el esfuerzo físico. Los cuartiles ($Q_1 = 114.68$, mediana = 128.71, $Q_3 = 143.59$) indican una distribución centrada y simétrica en torno al promedio, sin presencia de colas pronunciadas.

TABLE II: Estadística descriptiva de los datos de latidos por minuto.

Parámetro	Símbolo / Unidad	Valor
Tamaño muestral	n	127
Media	$\bar{x}$ [BPM]	129.16
Desviación estándar	s [BPM]	19.73
Mínimo	$x_{\min}$ [BPM]	90.44
Máximo	$x_{\max}$ [BPM]	164.85
Primer cuartil	Q_1 [BPM]	114.68
Mediana	Q_2 [BPM]	128.71
Tercer cuartil	Q_3 [BPM]	143.59

La prueba de normalidad de Anderson-Darling arrojó un valor de estadístico $A^2 = 0.5773$, el cual se comparó con los valores críticos correspondientes a distintos niveles de significancia. Para $\alpha = 0.05$, el valor crítico es 0.7640, por lo que se cumple $A^2 < A^2_{\text{crítico}}$. En consecuencia, no se rechaza la hipótesis nula (H_0) y se concluye que los datos provienen de una distribución normal con un nivel de confianza del 95 %.

El histograma de la Figura 2 muestra una distribución aproximadamente unimodal y simétrica, confirmando el resultado de

la prueba estadística. Por su parte, el diagrama de cajas de la Figura 3 evidencia una dispersión equilibrada alrededor de la mediana, sin observación de valores atípicos.

El test de Grubbs iterativo, aplicado con $\alpha = 0.05$, no detectó la existencia de valores fuera del rango normal, validando la calidad del conjunto de datos original y su idoneidad para el posterior proceso de filtrado. La ausencia de outliers garantiza que el ruido presente en la señal se distribuye de forma aleatoria, y no debido a errores de medición o adquisición.

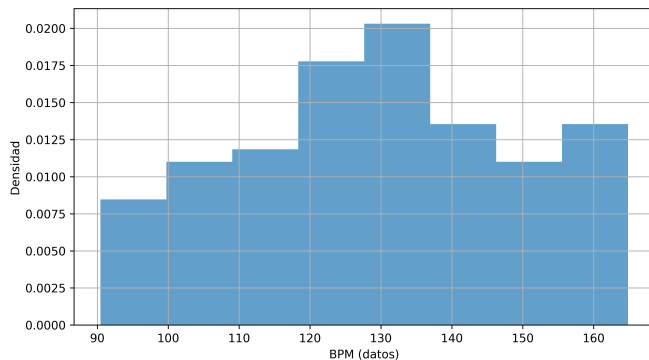


Fig. 2: Histograma de densidad para el conjunto de datos de BPM.

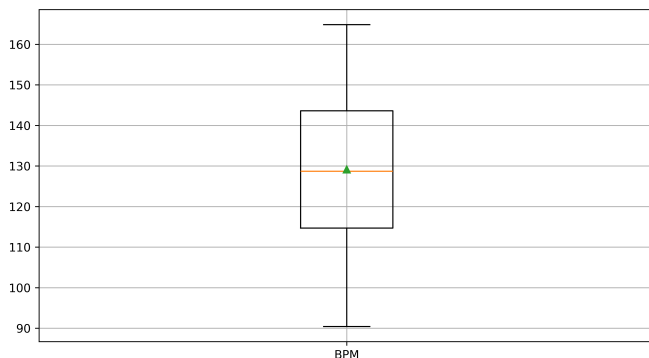


Fig. 3: Diagrama de cajas para el conjunto de datos de BPM.

Los resultados obtenidos permiten afirmar que el conjunto de datos inicial cumple con los supuestos de normalidad y ausencia de valores extremos, condiciones adecuadas para aplicar técnicas de filtrado lineal en la siguiente etapa de preprocesamiento.

2) Bloque Preprocesamiento

Como parte del proceso de preprocesamiento, se evaluaron los filtros de Mediana y Savitzky-Golay con el fin de determinar cuál ofrecía un mejor compromiso entre eficacia de filtrado, preservación de información y tiempo de ejecución.

La Figura 4 muestra la señal original obtenida del bloque Ecostress, donde pueden observarse pequeñas variaciones de alta frecuencia asociadas al ruido presente en la etapa de adquisición. Dichas oscilaciones justifican la necesidad de aplicar un método de filtrado digital que reduzca el ruido sin alterar las características fisiológicas de la señal.

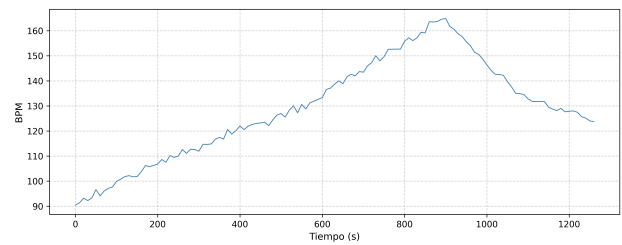


Fig. 4: Señal original de latidos por minuto antes del filtrado.

Con base en la matriz de comparación tipo *trade-off*, se seleccionaron los filtros de Mediana y Savitzky-Golay para realizar la evaluación final. El filtro de Mediana se implementó con una ventana de cinco muestras (`kernel_size=5`), mientras que el filtro de Savitzky-Golay se configuró con una ventana de cinco muestras y un polinomio de orden dos (`window_length=5`, `polyorder=2`).

La Figura 5 presenta la comparación entre la señal sin filtrar y la señal procesada mediante el filtro de Mediana. Este método eliminó eficazmente el ruido, manteniendo la forma general de la señal y sin introducir desplazamientos de fase.

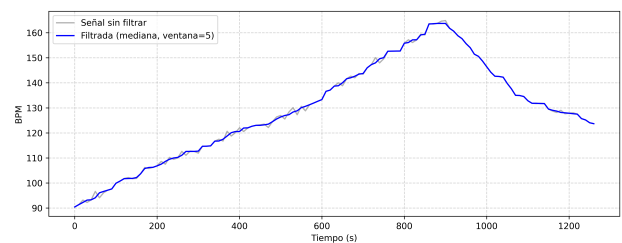


Fig. 5: Comparación entre la señal original y filtrada con el filtro de Mediana (ventana=5).

En la Figura 6 se muestra el resultado del filtrado con el método Savitzky-Golay, el cual produce una señal de salida más suave pero con una ligera pérdida de detalle en los picos de frecuencia cardíaca. Aunque este método conserva la morfología general, su mayor costo computacional lo hace menos eficiente para una implementación embebida.

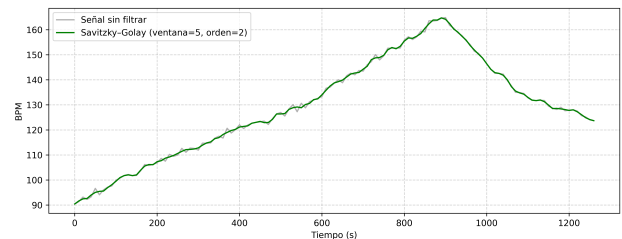


Fig. 6: Comparación entre la señal original y la señal filtrada con Savitzky-Golay (ventana=5, orden=2).

Los tiempos de ejecución se calcularon sobre 200 iteraciones consecutivas, utilizando el módulo `time.perf_counter()` de Python. La Tabla III muestra los resultados obtenidos, donde se observa que

ambos métodos cumplen ampliamente con el *deadline* de 10 s establecido para el procesamiento en tiempo real. No obstante, el filtro de Mediana resultó aproximadamente seis veces más rápido que Savitzky-Golay, manteniendo además una salida estable y libre de artefactos numéricos.

TABLE III: Comparación de desempeño entre los filtros evaluados.

Filtro	Ventana	Orden	Tiempo [ms]	Observaciones
Mediana	5	—	0.0661	Bajo costo computacional y alta estabilidad numérica.
Savitzky-Golay	5	2	0.6000	Suavizado eficaz, aunque con pérdida leve de picos.

A partir de los resultados, el filtro de Mediana fue seleccionado como método de preprocesamiento final para el sistema ECOSTRESS. Este filtro combina una excelente capacidad de supresión de ruido con una implementación sencilla y un tiempo de cómputo mínimo, lo cual lo hace idóneo para su portabilidad al SoC nRF52832 y su ejecución en tiempo real dentro del módulo transmisor (TX).

3) Bloque de Codificación

La Tabla IV muestra la comparación del desempeño de los códigos Hamming (7,4) y (15,11) obtenidos en las pruebas realizadas.

TABLE IV: Comparativa de desempeño entre los códigos Hamming.

Código	Bits de datos	Bits totales	Overhead
(7,4)	4	7	75%
(15,11)	11	15	36%

Los resultados muestran que el código (7,4) presenta una mayor redundancia, lo cual implica un menor rendimiento espectral pero una menor complejidad computacional. Por el contrario, el (15,11) ofrece una mejor eficiencia en la utilización del canal, manteniendo la capacidad de corrección de un bit. El análisis confirma que ambos esquemas cumplen su propósito de detección y corrección de errores, siendo el (15,11) más adecuado para sistemas donde la eficiencia de transmisión es prioritaria, mientras que el (7,4) puede ser preferido en implementaciones de baja complejidad o limitadas en recursos. Además, los tiempos de ejecución medidos fueron mínimos (del orden de milisegundos), lo que permite su integración en transmisores en tiempo real sin impacto significativo en latencia.

4) Bloque de Modulación

En la simulación de modulación se observaron las formas de onda de las señales NRZ, FSK y GFSK. La Figura 7 evidencia cómo la GFSK presenta una transición de fase más continua, evitando saltos abruptos y reduciendo la interferencia intersímbolo.

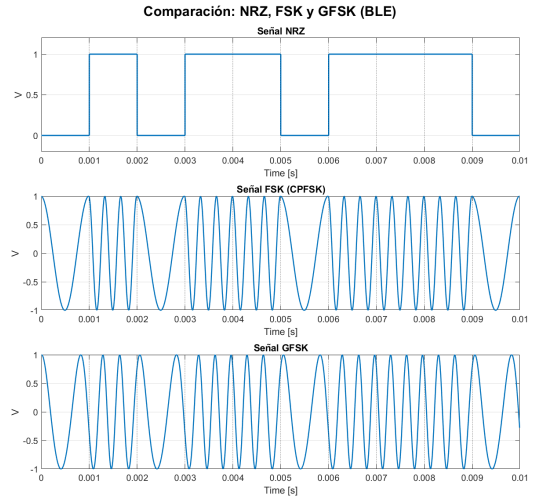


Fig. 7: Comparación entre señales NRZ, FSK y GFSK (BLE).

El filtrado gaussiano permitió suavizar la variación de frecuencia, cumpliendo con las condiciones del estándar BLE.

El tiempo de ejecución promedio fue de 0.1 s para 100 bits, lo que demuestra la eficiencia del algoritmo en MATLAB.

Comparando FSK y GFSK, se concluye que la segunda ofrece ventajas significativas en eficiencia espectral y estabilidad de fase, reduciendo la probabilidad de errores en entornos con ruido o atenuación de canal. La suavidad de la fase en GFSK implica también una menor ocupación de banda, lo que la hace adecuada para dispositivos de baja potencia como los basados en Bluetooth LE. En contraste, FSK sin filtrado presenta transiciones bruscas que amplían el espectro y aumentan la susceptibilidad a interferencias. Por tanto, la combinación GFSK + codificación Hamming constituye un esquema balanceado entre robustez, eficiencia y simplicidad de implementación.

5) Bloque Antena

- **Análisis del PCB y planos de referencia:** El circuito impreso presenta dos capas de cobre (Top y Bottom) sobre sustrato FR-4 de 1.6 mm de espesor. La capa superior (Top) contiene la mayor parte de las pistas de señal y potencia (VDD_P3V3), mientras que la capa inferior (Bottom) actúa como plano de tierra (GND) continuo. Ambas capas están interconectadas mediante vías pasantes. Esta configuración permite implementar una línea microstrip de 50 Ω que alimenta la antena desde la salida del módulo de RF.
- **Dispositivo de conmutación MM8130-2600:** El MM8130-2600 es un conector coaxial con switch interno de 50 Ω , tipo SWF, utilizado para seleccionar entre la antena integrada y un puerto externo de medición. Su rango operativo es de 0.1 a 6 GHz, con baja pérdida de inserción (0.35 dB a 2.4 GHz) y aislamiento de 25 dB [2]. Permite conectar equipos de laboratorio como un analizador vectorial de redes (VNA) para caracterizar la respuesta S_{11} o conectar una antena

externa certificada.

- **Selección de Antena:** Para el sistema BLE a 2.4 GHz se analizaron diferentes opciones basadas en la guía *UM10992* de NXP [3]. Por restricciones de espacio y

Opción	Eficiencia	Ancho de banda	Tamaño	Ganancia
Antena IFA	Media-alta	Media	Pequeño	1–2 dBi
Antena bow-shaped	Alta	Alta	Mediano	2–3 dBi
Antena tipo L	Media	Media	Pequeño	0–2 dBi

TABLE V:

Comparativa de opciones de antena para BLE 2.4 GHz

simplicidad de implementación sobre el PCB base, se seleccionó la antena chip SWRA092B, una *Inverted-F Antenna (IFA)* optimizada para la banda ISM de 2.400–2.4835 GHz, compatible con tecnologías BLE, ZigBee y Wi-Fi. Esta antena ofrece impedancia nominal de 50 Ω , ganancia de aproximadamente 1.5–2 dBi y eficiencia media, siendo adecuada para dispositivos compactos de baja potencia [4]. La implementación se ve en la Figura 8.

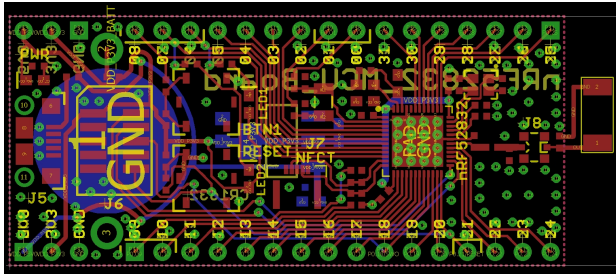


Fig. 8:

Implementación de la antena tipo IFA (SWRA092B) en el borde del PCB

B. Módulo Canal

Se verificó la generación de ruido AWGN mediante la transformación de Box–Muller, obteniendo una media de 0.0016 y varianza de 0.99791. Asimismo, se analizaron los resultados mediante un histograma de densidad de probabilidades, tal como se muestra en la Figura 9. Ambos resultados confirman su correspondencia con una distribución normal estándar.

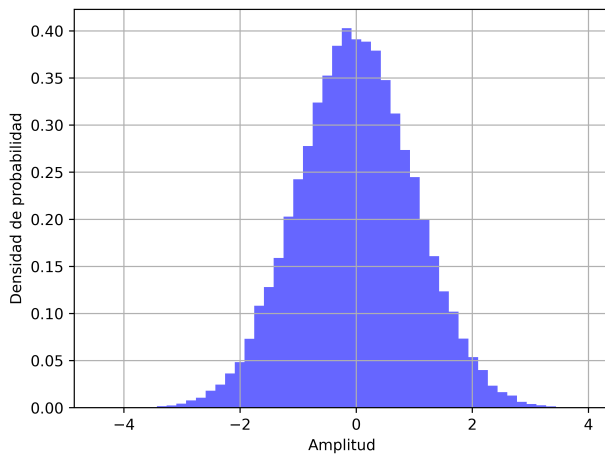


Fig. 9:

Histograma del ruido gaussiano blanco aditivo (AWGN) generado con Box-Muller

- Efecto de Fading en el Canal de Comunicación

El *fading* es un fenómeno propio de los canales de comunicación inalámbricos. Ocurre porque la señal transmitida no llega al receptor por un solo camino, sino por varios que se combinan entre sí. Cuando las ondas que recorren distintos trayectos se superponen, pueden reforzarse (interferencia constructiva) o cancelarse parcialmente (interferencia destructiva), lo que provoca aumentos o disminuciones momentáneas en la potencia recibida [5]. Dependiendo de la escala de tiempo y del entorno de transmisión, el *fading* puede clasificarse como de pequeña escala, donde las variaciones son rápidas y se asocian al movimiento del transmisor, del receptor o de objetos cercanos, o de gran escala, donde las variaciones se deben a pérdidas por distancia u obstáculos fijos.

En el contexto de la tecnología Bluetooth Low Energy (BLE), el *fading* puede causar atenuación severa y fluctuaciones en la relación señal-ruido (SNR), afectando la integridad del enlace y aumentando la probabilidad de error de bit (BER). Para mitigar este efecto, BLE incorpora técnicas de salto de frecuencia adaptativo (*Adaptive Frequency Hopping, AFH*), que permiten cambiar dinámicamente el canal de transmisión dentro de la banda de 2.4 GHz, reduciendo así la correlación temporal del canal y la susceptibilidad a zonas de desvanecimiento profundo. Además, la combinación de control de potencia y corrección de errores proporciona robustez adicional frente a las variaciones del canal. Este fenómeno es fundamental en la caracterización de canales reales y debe considerarse en el diseño de sistemas RF [6].

C. Módulo Receptor

1) Bloque Demodulador

2) Bloque Decodificador

Con el fin de evaluar el desempeño del bloque de decodificación, se compararon los bits codificados transmitidos con los bits demodulados recibidos para obtener una referencia de error previa a la corrección, y posteriormente se calculó la BER a nivel de información tras aplicar el decodificador Hamming (15,11). Además, se registró el número de errores corregidos por el algoritmo de síndrome para cuantificar su efectividad en el escenario de prueba.

TABLE VI: Resumen de resultados del decodificador Hamming (15,11)

Métrica	Valor
Bits TX codificados	1395
Bits RX demodulados	1395
Bits usados (múltiplo de 15)	1395
Palabras de código procesadas	93
Bits de información comparados	1023
Errores brutos (nivel código)	26
BER bruta (antes de decodificar)	1.864×10^{-2}
Errores corregidos por H(15,11)	20
Errores post-decodificador (info)	9
BER a nivel de información	8.798×10^{-3}
Porcentaje de errores corregidos	76.92%

A partir de los datos de la Tabla VI, se observa coherencia en la segmentación de bloques, ya que los 1395 bits utilizados corresponden exactamente a 93 palabras de código de 15 bits y, por ende, a 1023 bits de información recuperados (93×11). Esto confirma que el procesamiento por palabras completas se realizó de forma correcta y consistente con la estructura del código.

En cuanto al desempeño del decodificador, la BER bruta previa a la corrección fue de aproximadamente 1.86%, mientras que la BER a nivel de información después de la decodificación se redujo a aproximadamente 0.88%. Esta disminución equivale a una mejora de alrededor de 2.1 veces en la tasa de error a nivel de información, lo que evidencia el aporte del código Hamming (15,11) en la mitigación de errores introducidos por el canal.

Adicionalmente, el decodificador corrigió 20 de los 26 errores detectados a nivel de bits de código, lo que corresponde a una tasa de corrección cercana al 76.92%. La presencia de errores residuales tras la decodificación es consistente con las limitaciones teóricas del código Hamming, dado que este esquema corrige de manera confiable errores de un solo bit por palabra, pero no garantiza corrección cuando ocurren múltiples errores dentro del mismo bloque.

Finalmente, como salida funcional del bloque, los bits de información corregidos fueron almacenados en `bits_decodificados_h1511.txt`, permitiendo su uso directo en etapas posteriores del sistema o en validaciones cruzadas con MATLAB. En paralelo, las estadísticas de desempeño se registraron en `resultados_decodificador_h1511.txt` para documentar formalmente el comportamiento del decodificador bajo el E_b/N_0 considerado en esta prueba.

3) Bloque Visualización

La Figura 10 presenta la curva de tasa de error de bit en función de la SNR para una referencia teórica de BPSK en canal AWGN. Se observa una disminución monótona de la BER a medida que aumenta la SNR, con valores del orden de 10^{-1} para $SNR = 0$ dB y del orden de 10^{-5} para $SNR = 10$ dB. Esta curva sirve como línea base para comparar el desempeño de los esquemas codificados Hamming (7, 4) y Hamming (15, 11).

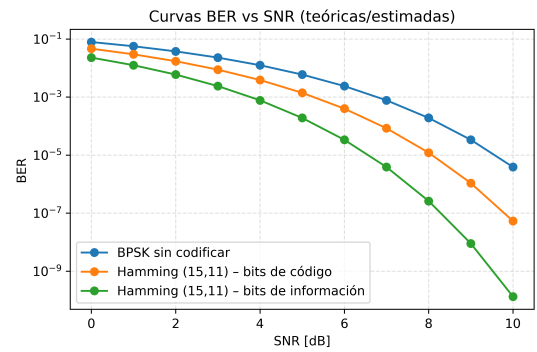


Fig. 10: Curvas BER vs. SNR para la referencia BPSK sin codificar

La Figura 11 muestra la comparación entre la señal de BPM de referencia, obtenida a partir del archivo `Dataset_Crudo.xlsx`, y la señal reconstruida tras el enlace, almacenada en `Dataset.xlsx`. Ambas curvas presentan un seguimiento muy cercano a lo largo de todo el recorrido temporal, con diferencias puntuales de pequeña magnitud. A partir de estas series se obtiene un error cuadrático medio de $RMSE = 0.58$ bpm y un sesgo porcentual de $Pbias = 0.40$ %. Estos valores indican que el sistema de comunicación reproduce la dinámica de la frecuencia cardíaca con un error absoluto medio inferior a 1 bpm y sin un sesgo sistemático significativo, lo cual resulta adecuado para aplicaciones de monitoreo de entrenamiento.

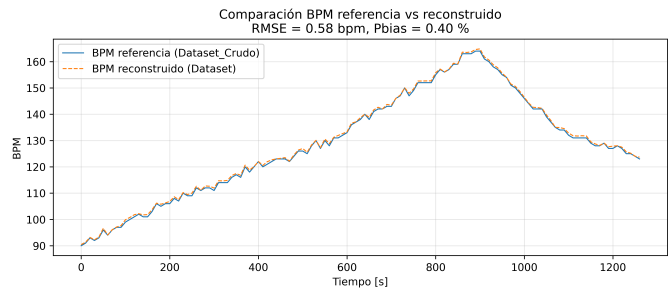


Fig. 11: Comparación entre la señal de BPM de referencia y la señal reconstruida por el sistema de telemetría

La Figura 12 ilustra la clasificación de la señal reconstruida en zonas de entrenamiento en función de la frecuencia cardíaca relativa a $FC_{max} = 188$ bpm. Cada muestra se colorea según la zona correspondiente, evidenciando una transición progresiva desde valores próximos al calentamiento y quema de grasa hasta regiones aeróbicas y anaeróbicas en la parte central de la sesión, seguida de una fase de descenso nuevamente hacia intensidades moderadas. Esta representación confirma que el sistema conserva la estructura global de las zonas de entrenamiento definidas a partir de la señal de referencia y permite interpretar de forma intuitiva el esfuerzo realizado durante la actividad.

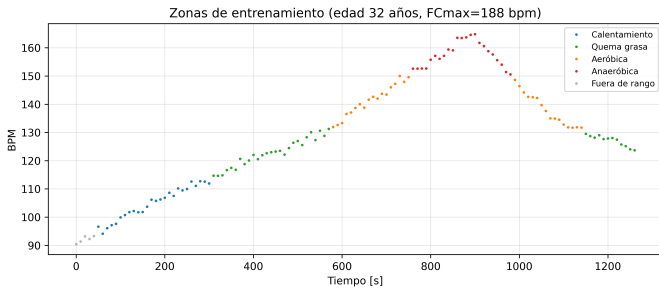


Fig. 12: Clasificación de la señal reconstruida en zonas de entrenamiento en función del porcentaje de FC_{max}

La medición del tiempo de ejecución del bloque de visualización arroja un tiempo promedio de $t_{prom} = 1,1608$ ms con una desviación estándar de $\sigma_t = 0,2686$ ms para $N = 200$ ejecuciones consecutivas. Estos resultados muestran que la generación de tablas y figuras presenta un coste computacional muy reducido frente al resto de bloques del sistema, por lo que la visualización puede ejecutarse de manera reiterada durante la etapa de validación y, si fuera necesario, integrarse en flujos de análisis casi en tiempo real.

IV. CONCLUSIONES

El análisis estadístico del conjunto de datos de la prueba de Ecostress confirmó que la señal de latidos por minuto sigue una distribución normal con un nivel de confianza del 95 %, sin presencia de valores atípicos. Estos resultados validan la calidad del conjunto de datos y garantizan que puede emplearse como entrada confiable para las etapas de preprocesamiento y codificación del sistema transmisor.

De la evaluación comparativa entre los filtros de Mediana y Savitzky-Golay se determinó que el primero ofrece la mejor relación entre desempeño y costo computacional. Con un tiempo promedio de 0.066 ms por iteración y una alta capacidad de eliminación de ruido impulsivo, el filtro de Mediana resulta óptimo para su implementación en el módulo transmisor del sistema ECOSTRESS, cumpliendo con los requerimientos de operación en tiempo real del SoC nRF52832.

La codificación Hamming (7,4) y (15,11) implementada en Python validó la corrección de un bit con baja complejidad y tiempos mínimos. El esquema (15,11) destacó por su menor redundancia (36 %) y mayor eficiencia. La modulación GFSK desarrollada en Octave/MATLAB reprodujo el comportamiento del estándar BLE, mostrando fase continua y menor ancho de banda que la FSK. En conjunto, ambos bloques demostraron un transmisor robusto, eficiente y adecuado para aplicaciones inalámbricas de baja potencia.

El bloque antena cumple la función de enlazar el transmisor BLE con el espacio libre mediante una red de 50Ω controlada. El uso del conector MM8130-2600 facilita la conmutación entre modo de operación y modo de prueba, mientras que la

antena SWRA092B proporciona una solución compacta y de fácil integración.

El modelado del canal mediante ruido AWGN permitió representar de forma aproximada las perturbaciones que afectan la transmisión inalámbrica en un entorno BLE. La implementación del generador de ruido con la transformación de Box-Muller demostró un comportamiento gaussiano con media cero y varianza unitaria.

Con los resultados obtenidos se concluye que el decodificador Hamming (15, 11) fue implementado correctamente y cumple su función de corrección de errores. Esto se evidencia en la reducción de la BER desde 1.864×10^{-2} (aprox. 1.86% de error antes de decodificar) hasta 8.798×10^{-3} (aprox. 0.88% tras la decodificación). Además, el algoritmo logró corregir 20 de 26 errores detectados a nivel de bits de código, equivalente a un 76.92% de corrección. Por tanto, el uso de Hamming (15, 11) mejora de forma clara la confiabilidad del enlace en este escenario de prueba, aunque persisten errores residuales consistentes con la limitación del código ante errores múltiples en una misma palabra.

REFERENCES

- [1] IEEE Standard 802.15.1-2005: *Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Wireless Personal Area Networks (WPANs)*, IEEE Std., 2005.
- [2] Murata Electronics, "Mm8130-2600ra2 rf connector," <https://www.digikey.com/en/products/detail/murata-electronics/MM8130-2600RA2/1775922>, 2024, consultado en octubre de 2025.
- [3] NXP Semiconductors, *UM10992: LPCXpresso55S69 Board User Guide*, <https://www.nxp.com/docs/en/user-guide/UM10992.pdf>, 2023, consultado en octubre de 2025.
- [4] Texas Instruments, *SWRA092B: Antenna Selection Guide*, <https://www.ti.com/lit/an/swra092b/swra092b.pdf>, 2015, consultado en octubre de 2025.
- [5] S. Cloudt, "Characterization of the 2.4 ghz ism band wireless channel for bluetooth low energy applications," Master's thesis, Eindhoven University of Technology (TU/e), 2021, master's Thesis in Electrical Engineering. Consultado en octubre de 2025.
- [6] J. F. N. Molina, "Diseño y simulación de antenas de microcinta para aplicaciones bluetooth y wi-fi en la banda ism de 2.4 ghz," Master's thesis, Universidad Politécnica de Cartagena (UPCT), 2019, trabajo Fin de Grado en Ingeniería de Telecomunicaciones. Consultado en octubre de 2025.
- [7] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.
- [8] Bluetooth SIG, *Bluetooth Core Specification Version 5.4*, Bluetooth Special Interest Group, Feb. 2023, available online. [Online]. Available: <https://www.bluetooth.com/specifications/bluetooth-core-specification/>
- [9] P. L'Ecuyer, *Simulation of Modern Communication Systems*. Cham, Switzerland: Springer, 2021.
- [10] MathWorks, *Gaussian Frequency Shift Keying (GFSK) Modulation*, MathWorks Inc., 2024, accessed 2024. [Online]. Available: <https://www.mathworks.com/help/comm/ref/gfskmod.html>
- [11] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Prentice Hall, 2001.
- [12] S. Haykin, *Communication Systems*, 5th ed. Wiley, 2008.
- [13] MathWorks, *Fading Channels (Communications Toolbox)*, <https://www.mathworks.com/help/comm/ug/fading-channels.html>, 2025, consultado en octubre de 2025.